15 class-validator AND class-transformer

These are two separate npm packages that Nest's ValidationPipe uses TOGETHER to turn a raw, untyped JSON request body into a trusted, validated DTO instance. They do different jobs:

class-transformer	→ converts a PLAIN OBJECT into a REAL CLASS INSTANCE (and back). Shape/conversion.
class-validator	→ runs VALIDATION RULES (decorators) against a class instance, and reports errors. Correctness.

```
Raw JSON body -> class-transformer (plainToInstance) -> DTO instance
                                                            |
                                                  class-validator (validate)
                                                            |
                                                  valid? -> handler runs
                                                  invalid? -> 400 thrown
```

WHERE THEY'RE USED IN THIS PROJECT

```ts
// main.ts
app.useGlobalPipes(new ValidationPipe());
```

That one line wires BOTH packages into every request, globally. You never call plainToInstance() or validate() yourself — ValidationPipe does it for you, automatically, for every @Body()/@Query()/@Param() that is typed as a DTO class.

class-validator — VALIDATION RULES AS DECORATORS

```ts
// src/task/dto/create-task.dto.ts
import { IsBoolean, IsNotEmpty, IsOptional, IsString } from 'class-validator';

export class CreateTaskDto {
  @IsNotEmpty()    // title is required and cannot be empty
  @IsString()      // title must be a string
  title!: string;

  @IsOptional()           // description is not required
  @IsString()             // if provided, description must be a string
  description?: string;

  @IsOptional()        // done is not required
  @IsBoolean()         // if provided, done must be true or false
  done?: boolean;
}
```

Each decorator attaches a runtime check to that property:

```ts
@IsString()      value must be typeof 'string'
@IsNotEmpty()    value must not be '', null, or undefined
```

```
@IsOptional()    SKIPS all other decorators on this property if the
                 value is undefined — this is why description/done
                 don't also need @IsNotEmpty(); without @IsOptional(),
                 @IsBoolean() alone would reject a missing 'done'.
@IsBoolean()     value must be a real boolean (true/false)
```

A decorator order note: stacking multiple decorators on one property (as above) runs them all — order between @IsString()/@IsNotEmpty() doesn't matter here, but @IsOptional() short-circuiting the rest is why it's commonly listed first by convention.

MORE COMMON class-validator DECORATORS

@IsNumber()	@IsInt()	@Min(0) / @Max(100)
@IsEmail()	@IsUUID()	@IsDateString()
@IsEnum(SomeEnum)	@IsArray()	@ArrayMinSize(1)
@Length(1, 50)	@IsPositive()	@Matches(/regex/)
@ValidateNested()	used with @Type() (below) to validate an object/array PROPERTY that is itself a DTO.	

WHAT HAPPENS WHEN VALIDATION FAILS

```
POST /task { "description": "no title" }
-> class-validator collects every broken rule into an array of
   error messages
-> ValidationPipe wraps them and throws a BadRequestException
-> Nest turns that into an HTTP 400, before addProduct()/create()
   or any handler code ever runs:

{
  "message": ["title should not be empty", "title must be a string"],
  "error": "Bad Request",
  "statusCode": 400
}
```

class-transformer — SHAPING THE OBJECT ITSELF

class-validator's decorators only work on a REAL CLASS INSTANCE (not a plain object) — decorators need an actual class to attach metadata to. But everything arriving over HTTP is just parsed JSON: a plain object with no class identity at all. class-transformer bridges that gap.

```
import { plainToInstance } from 'class-transformer';

const raw = { title: 'Write notes' };            // plain object
const dto = plainToInstance(CreateTaskDto, raw);  // now a REAL
                                                    // CreateTaskDto
                                                    // instance
dto instanceof CreateTaskDto;  // true
```

ValidationPipe calls plainToInstance() internally, every single request — that's the step that makes class-validator's decorators usable at all on data that started life as JSON.

class-transformer ALSO has its own decorators, for controlling that conversion:

@Type(() => Date)	convert a string/number into a real Date
@Type(() => AddressDto)	convert a nested object into a nested DTO instance (pairs with @ValidateNested() on the same property, for validating objects-within-objects/arrays)
@Expose() / @Exclude()	control which properties survive when converting a class instance BACK to a plain object (e.g. hiding a password field when returning a User entity as a response)
@Transform(fn)	run arbitrary custom conversion logic

THE ValidationPipe `transform` OPTION — WORTH KNOWING

This project's main.ts currently does:

```
app.useGlobalPipes(new ValidationPipe());
```

That's the DEFAULT config, which means `transform: false`. Concretely, that means:

- class-validator's checks still run correctly (validation always needs a real instance internally, regardless of this option).
- BUT the plain object — not the transformed DTO instance — is what actually reaches your handler. So `dto instanceof CreateTaskDto` is FALSE inside task.controller.ts, and any @Type()-based coercion (e.g. converting a numeric query string to a real number) does NOT carry through to the value your handler receives.

Passing `{ transform: true }` changes that:

```
app.useGlobalPipes(new ValidationPipe({ transform: true }));
```

- The actual transformed instance (with any @Type() conversions applied) is what gets injected into the handler.
- This is also what lets @Query('page') be auto-converted from a URL string to a number without a separate ParseIntPipe, IF the DTO field is typed as `number` and decorated appropriately — this project currently does that conversion manually per-param instead, e.g. @Param('id', ParseIntPipe) id: number (see theory/09-pipes.txt).

Two more commonly-paired options:

whitelist: true	strips any properties NOT declared on the DTO from the incoming object (e.g. a client sneaking in an extra 'isAdmin' field gets it silently dropped)
forbidNonWhitelisted: true	same detection, but throws a 400 instead of silently stripping

class-validator vs class-transformer — QUICK DISTINCTION

```
class-validator     Answers: "is this data VALID?"
                    Decorators: @IsString(), @IsNotEmpty(), @Min()...
                    Throws/reports errors.
```

```
class-transformer   Answers: "what SHAPE/TYPE should this data be?"
                    Decorators: @Type(), @Expose(), @Exclude(),
                    @Transform()
                    Converts data, doesn't judge it.
```

They're independent packages — you could use class-transformer alone (just shaping objects, no validation) or class-validator alone (if you already have real class instances from somewhere other than JSON). Nest's ValidationPipe is what wires them together specifically for the "HTTP request body → trusted DTO" pipeline.

★ RULE OF THUMB

Any field a DTO needs to accept from a client gets a class-validator decorator (@IsString(), @IsOptional(), etc.) so ValidationPipe can reject bad requests automatically. Reach for class-transformer's own decorators (@Type(), @Expose()/@Exclude()) only when the shape itself needs converting — nested DTOs, dates, or controlling what a response object exposes — not for plain presence/type checks, which class-validator already covers.

See also: theory/13-dto.txt, theory/09-pipes.txt, theory/14-interface-vs-dto.txt